

VORWORT

partially yoinked from Jasmin Fässler, Nina Grässli & Jannis Tschan, motivated through Ian MacKaye

TODO:

- Beispiele:
- Ist Grammatik eindeutig? (FS_2026)5)
- Parse-tree finden (FS_2022)5))
- Prüfungsaufgaben:
- HS_2021)4)
- FS_2022)4)

VORGEHEN ZU AUFGABENTYPEN

Frage	Antwort
Ist Sprache regulär?	Ja: DEA / NEA / RegEx Nein: Pumping Lemma
Ist Sprache kontextfrei?	Ja: Stackautomat / CNF / BNF Nein: Pumping Lemma (CFL)
Turing-Maschinen	Berechnungsgeschichte
Ist eine Sprache Turing-Vollständig?	Ja: TM-Simulation Nein: Halteproblem
Kann Problem von einem Computer gelöst werden?	Reduktion auf Halteproblem
Kann ein Programm entscheiden, ob zulässige Inputs eine Eigenschaft erfüllen?	Satz von Rice
Kann eine nichtdeterministische Maschine ... entscheiden?	NP-Verifizierer
Warum kann ein Problem nicht gelöst werden?	Reduktion auf NP-Vollständiges Problem

DETERMINISTISCHE ENDLICHE AUTOMATEN (DEA)

$A = (Q, \Sigma, \delta, q, F)$

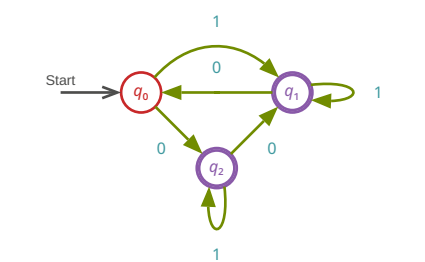
$Q = \{q_0, q_1, \dots, q_n\}$

Alphabet: Σ

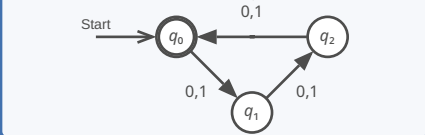
Übergangsfunktion: $\delta: Q \times \Sigma \rightarrow Q$

Startzustand: $q \in Q$

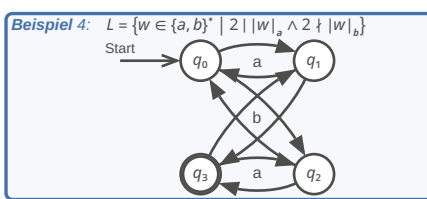
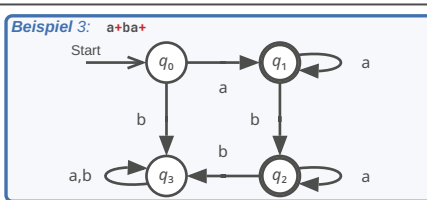
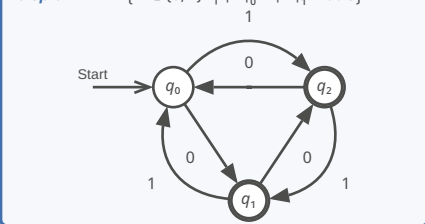
Akzeptierzustände: $F \subseteq Q$



Beispiel 1: $L = \{w \in \{0, 1\}^* \mid |w|_0 - |w|_1 \equiv 0 \pmod 3\}$



Beispiel 2: $L = \{w \in \{0, 1\}^* \mid |w|_0 \equiv |w|_1 \pmod 3\}$

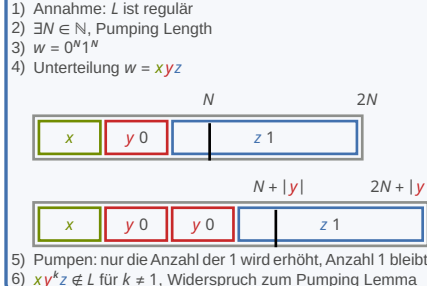


PUMPING LEMMA (DEA)

Um zu beweisen, dass eine Sprache **nicht** regulär ist

- Vorgehen:
- 1) Annahme: L ist regulär
 - 2) Gemäss Pumping Lemma gibt es die Pumping Length N – Es darf keine Annahme über die konkrete Grösse von N gemacht werden!
 - 3) Wähle ein Wort $w \in L$ mit $|w| \geq N$ – Die Definition **muss** N verwenden!
 - 4) Aufteilung des Wortes gemäss Pumping Lemma – $w = xyz$, $|xy| \leq N$, $|y| > 0$
 - 5) Auswirkung des Pumpens – $xy^kz \notin L$ für mindestens ein $k \in \mathbb{N}$ (mit Begründung!)
 - 6) Widerspruch und Schlussfolgerung, dass die Annahme nicht zutreffen kann

Beispiel 5: $L = \{0^n 1^n \mid n \geq 0\}$ ist **nicht** regulär



Beispiel 6: $L =$ alle korrekte Integralausdrücke

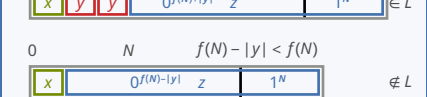
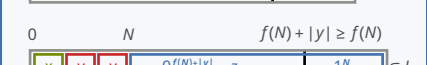
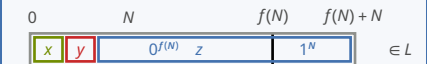
- 1) Annahme: L ist regulär
- 2) $\exists N \in \mathbb{N}$, Pumping Length
- 3) $w = \int_{-1}^1 \int_{-1}^1 \dots \int_{-1}^1 dx_1 \dots dx_n \in L$
- 4) Unterteile $w = xyz$, wobei $|y| \geq 1 \wedge |xy| \leq N \wedge xy^kz \in L$
- 5) Pumpen: Der Teil xy enthält keine dx_k , beim pumpen nimmt also nur die Anzahl der Integralzeichen zu, so dass kein korrekter Integralausdruck stehen bleibt.
- 6) Der Widerspruch zeigt, dass die Annahme, L sei regulär, nicht haltbar ist. Also ist L nicht regulär.

6 Schritte des Pumping-Lemma-Beweises: Annahme (A) 1 Punkt, Pumping Length (N) 1 Punkt, Wort (W) 1 Punkt, Unterteilung (U) 1 Punkt, Widerspruch beim Pumpen (P) 1 Punkt, Folgerung (F) 1 Punkt.

Beispiel 7: $L = w \in \{0, 1\}^* \mid |w|_0 \geq f(|w|_1)$

Sei $f: \mathbb{N} \rightarrow \mathbb{N}$ eine streng monoton wachsende Funktion. Eine solche Funktion nimmt beliebig grosse Werte an und es gilt $f(n) \geq n$ für alle $n \in \mathbb{N}$.

- 1) Annahme: L ist regulär.
- 2) Nach dem Pumping Lemma gibts die Pumping Length N
- 3) Wir wählen das Wort $0^{f(N)} 1^N \in L$.
- 4) Nach dem Pumping Lemma lässt sich das Wort w in drei Teile $w = xyz$ zerlegen mit $|xy| \leq N$ und $|y| > 0$. Weil $f(N) \geq N$ enthält y ausschliesslich Nullen.



- 5) Beim Aufpumpen wird die Anzahl der Nullen grösser, das bedeutet aber nicht, dass das Wort nicht mehr in der Sprache enthalten ist, da nur $|w|_0 \geq f(|w|_1)$ verlangt ist. Beim Abpumpen wird allerdings die Anzahl der Nullen kleiner, xz enthält nur noch $f(N) - |y|$ Nullen. Damit haben wir $|xz|_0 = f(N) - |y| < f(N) = |xz|_1$, das Wort xz ist also nicht mehr in L , im Widerspruch zur Aussage des Pumping Lemmas.
- 6) Der Widerspruch zeigt, dass Annahme, L sei regulär, nicht haltbar ist. Also ist L nicht regulär.

NICHTDETERM. ENDLICHE AUTOMATEN (NEA)

$A = (Q, \Sigma, \delta, q, F)$

– Endliche Menge von Zuständen: Q

– Alphabet: Σ

– Übergangsfunktion: $\delta: Q \times \Sigma \rightarrow P(Q)$

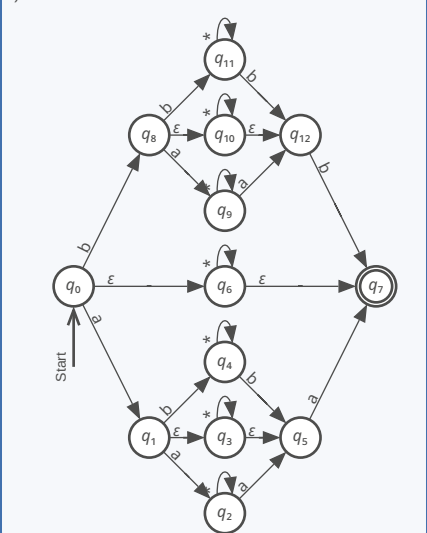
– Startzustand: $q \in Q$

– Akzeptierzustände: $F \subseteq Q$

Beispiel 8: $L = \{wxw^t \mid w, x \in \Sigma^* \wedge |w| \leq 2\}, \Sigma = \{a, b\}$

Das Wort w^t ist die gespiegelte Version des Wortes w . Ist die Sprache L regulär?

- Ja, alle diese Beweise sind gültig:
- 1) $w = \epsilon$ darf sein und daher kann $w = x \in \Sigma^*$ sein. Es folgt $L = \Sigma^*$, diese Sprache ist also regulär
 - 2) Regex: $aa^*aa^*|ab^*ba^*|ba^*ab^*|bb^*bb^*|a^*a|b^*b|a^*b|b^*a$
 - 3) NEA:



VERALLGEMEINERTER NEA (VNEA)

Begriff	Definition
Regulär	Es gibt einen DEA A , der L akzeptiert, also $L(A) = L$
Regulärer Ausdruck	Zeichenkette r zur Beschreibung einer regulären Sprache $L = L(r)$
Reguläre Operationen	$L(r_1) \cup L(r_2) = L(r_1 r_2)$ $L(r_1)L(r_2) = L(r_1 r_2)$ $L(r_1)^* = L(r_1^*)$
Verallgemeinerter NEA	NEA_v , dessen Übergänge mit regulären Ausdrücken beschriftet sind

PRIMITIVE REGULÄRE AUSDRÜCKE

Reguläre Ausdrücke für Wörter mit Länge ≤ 1

$L = L(r)$	r	NEA
$\emptyset$	$\emptyset$	Start $\rightarrow$
$\{\epsilon\}$	$\{\epsilon\}$	Start $\rightarrow$
$\{a\}$	a	Start $\rightarrow$
$\{o, s, t\}$	$[ost]$	Start $\rightarrow$
$\{a, b, \dots, s\}$	$[a-s]$	Start $\rightarrow$
Σ	$\cdot$	Start $\rightarrow$

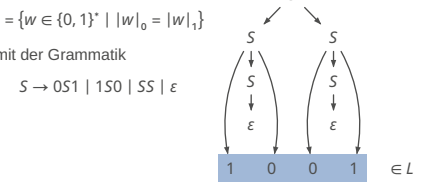
KONTEXTFREIE SPRACHEN (CFL)

Kontextfrei: L kann nur von einem ND PDA erkannt werden

$G = (V, \Sigma, R, S)$

- V : Variablen
- Σ : Terminalsymbole (Alphabet)
- R : Regeln der Form $A \rightarrow x_1 x_2 \dots x_n$ mit $A \in V, x_i \in V \cup \Sigma$
- S : Startvariable

Parse Tree



Beispiel 9: Ist L kontextfrei?

$L = \{wuw^t \mid w, u \in \Sigma^* \wedge |u| \leq 3\}, \Sigma = \{a, b\}$

(w^t ist das gespiegelte Wort von w)

Die Sprache ist kontextfrei, weil die folgende kontextfreie Grammatik die Sprache erzeugen kann:

$S \rightarrow aSa \mid bSb \mid U$
 $U \rightarrow \epsilon \mid A \mid AA \mid AAA$
 $A \rightarrow a \mid b$

Alternativ kann man auch einen Stackautomaten angeben, der die Sprache akzeptiert.

Grammatik für A (A) 1 Punkt, Schachtelungs-idee (S) 2 Punkt, der mittlere Teil kann leer sein (U) 1 Punkt, der mittlere Teil kann aus a und b bestehen (B) 1 Punkt, Schlussfolgerung kontextfrei (K) 1 Punkt.

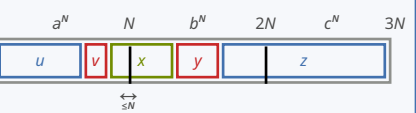
PUMPING LEMMA (CFL)

Um zu beweisen, dass eine Sprache **nicht** kontextfrei ist

- Voraussetzungen:
- 1) $|v| > 0$
 - 2) $|vxy| \leq N$
 - 3) $\forall k \in \mathbb{N}. (uv^kxy^kz \in L)$

Beispiel 10: $\{a^n b^n c^n \mid n \geq 0\}$

Wort: $w = a^n b^n c^n$



Beim Pumpen nimmt die Anzahl der a und b zu, nicht aber die Anzahl der $c \Rightarrow \forall k \neq 1. (uv^kxy^kz \notin L)$

Beispiel 11: $L = \{w \in \{0, 1\}^* \mid w = 0^k 1^k 0^k 1^k\}$

- 1) Annahme: L ist kontextfrei
- 2) Nach dem Pumping Lemma $\exists N \in \mathbb{N}$, Pumping Length
- 3) Wähle das Wort $w = 0^{N^2} 1^{N^2} 0^{N^2} 1^{N^2}$
- 4) Nach dem Pumping Lemma gibt es eine Aufteilung von w in fünf Teile $w = uvxyz$ derart, dass $|vxy| \leq N \wedge |vy| \geq 1$. Ausserdem ist jedes gepumpte Wort $uv^kxy^kz \in L$.
- 5) Da sich die Anzahl der Einsen beim Pumpen nicht ändern darf, müssen v und y vollständig in einem Nullen-Block enthalten sein. Daher kann sich nur die Anzahl der Nullen in höchstens zwei Nullen-Blöcken ändern. Die beiden Blöcke müssen wegen $|vxy| \leq N$ ausserdem benachbart sein. Zum ersten Nullen-Block gehört der dritte, der gleich viele Nullen enthalten muss, zum zweiten gehört der vierte. Wie auch immer die beiden Blöcke gewählt werden, ändert sich die Anzahl Nullen in den Blöcken, aber nicht in den zugehörigen Blöcken. Das gepumpte Wort kann also nicht mehr in L sein.
- 6) Dieser Widerspruch zeigt, dass die Annahme, L sei kontextfrei, nicht haltbar ist. Also ist L nicht kontextfrei.

Pumping Lemma und Annahme L kontextfrei (PL) 1 Punkt, Pumping Length (N) 1 Punkt, Wahl eines Wortes (W) 1 Punkt, Unterteilung (U) 1 Punkt, Widerspruch beim Pumpen (P) 1 Punkt, Schlussfolgerung (S) 1 Punkt.

CHOMSKY-NORMALFORM (CNF)

- S kommt auf der rechten Seite nicht vor
- Jede Regel von der Form $A \rightarrow BC$ oder $A \rightarrow a$
- $S \rightarrow \epsilon$ ist erlaubt

UMWANDLUNG

- 1) Neue Startvariable $S_0 \rightarrow S$ (wenn nötig)
 - 2) ϵ -Regeln:
- $A \rightarrow \epsilon \Rightarrow \left\{ \begin{array}{l} B \rightarrow AC \\ B \rightarrow AC \end{array} \right\}$

- 3) Unit-Rules:
- $A \rightarrow B \Rightarrow \left\{ \begin{array}{l} A \rightarrow CD \\ B \rightarrow CD \end{array} \right\}$

- 4) Verkettungen:
- $A \rightarrow u_1 u_2 \dots u_n$
 $A \rightarrow u_1 A_1, A_1 \rightarrow u_2 A_2, \dots, A_{n-2} \rightarrow u_{n-1} u_n$
falls u_i ein Terminalsymbol: $A_{i-1} \rightarrow u_i A_i, u_i \rightarrow u_i$

Beispiel 12: $S \rightarrow ASA \mid aB, A \rightarrow B \mid S, B \rightarrow b \mid \varepsilon$

1) Startvariable

$S_0 \rightarrow S$
 $S \rightarrow ASA \mid aB$
 $A \rightarrow B \mid S$
 $B \rightarrow b \mid \varepsilon$

2) ε -Regel

$S_0 \rightarrow S$
 $S \rightarrow ASA \mid aB \mid a$
 $A \rightarrow B \mid \varepsilon \mid S$
 $B \rightarrow b$

3) Unit-Regel

$S_0 \rightarrow ASA \mid SA \mid AS \mid aB \mid a$
 $S \rightarrow ASA \mid SA \mid AS \mid aB \mid a$
 $A \rightarrow B \mid ASA \mid SA \mid AS \mid aB \mid a$
 $B \rightarrow b$

4) Komplexe Regeln

$S_0 \rightarrow AA_1 \mid SA \mid AS \mid UB \mid a$
 $S \rightarrow AA_1 \mid SA \mid AS \mid UB \mid a$
 $A \rightarrow b \mid AA_1 \mid SA \mid AS \mid UB \mid a$
 $A_1 \rightarrow SA$
 $U \rightarrow a$
 $B \rightarrow b$

ABLEITUNGSDREIECK

Ideen

- Parse Tree aus $A \rightarrow BC$ und $T \rightarrow t$
- Variablen in Tabelle füllen

Prinzip

- Einem Teilwort entspricht ein Feld der Tabelle (rot hinterlegt)
- Das Feld wird mit den Variablen gefüllt, aus denen das Teilwort abgeleitet werden kann.

Beispiel

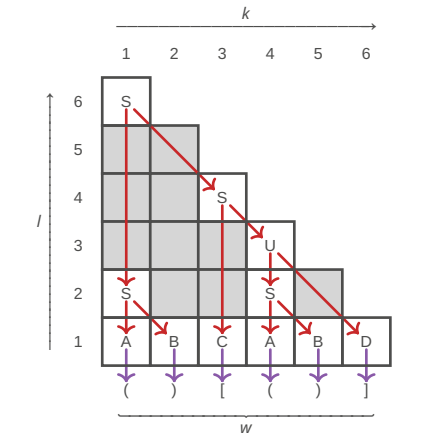
Parsen des Wortes $()[(())]$

$S \rightarrow AB \mid CD \mid CU \mid SS$
 $U \rightarrow SD$
 $A \rightarrow ($

$B \rightarrow)$
 $C \rightarrow [$
 $D \rightarrow]$

Definitionen

Die Felder (k, l_1) und $(k + l_1, l - l_1)$ heissen **korrespondierende Felder** im Ableitungsdreieck des Feldes (k, l) .



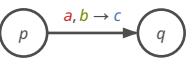
BNF

- Variablen: **<variablen-name>**
- Einzelne Zeichen: A
- Zeichenketten: 'BEISPIEL'
- Regeln: **<variablen-name> ::= Ausdruck**
- Ausdrücke sind Folgen von Variablen, einzelnen Zeichen oder Zeichenketten, getrennt durch |

STACKAUTOMAT (PDA)

$P = (Q, \Sigma, \Gamma, \delta, q_0, F)$

- Q: Zustände
- Σ : Eingabe-Alphabet
- Γ : Stack-Alphabet
- $\delta: Q \times \Sigma \times \Gamma \rightarrow P(Q \times \Gamma)$
- $q_0 \in Q$: Startzustand
- $F \subset Q$: Akzeptierzustände



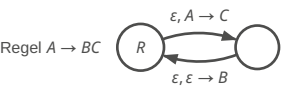
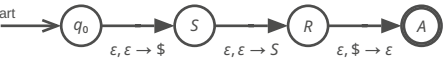
a vom Input

b vom Stack entfernen (Bedingung)

c auf den Stack

GRAMMATIK ABLESEN

Ist L eine kontextfreie Sprache, dann gibt es einen Stackautomaten P, der L akzeptiert, $L = L(P)$.



TURING MASCHINEN

(deterministische) Turing-Maschine

$M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$

- Q: Zustände
- Σ : Alphabet
- Γ : Bandalphabet, $_ \in \Gamma \setminus \Sigma$
- $\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$
- $q_0 \in Q$: Startzustand
- $q_{\text{accept}} \in Q$: Akzeptierzustand
- $q_{\text{reject}} \in Q$: Ablehnzustand, $q_{\text{accept}} \neq q_{\text{reject}}$

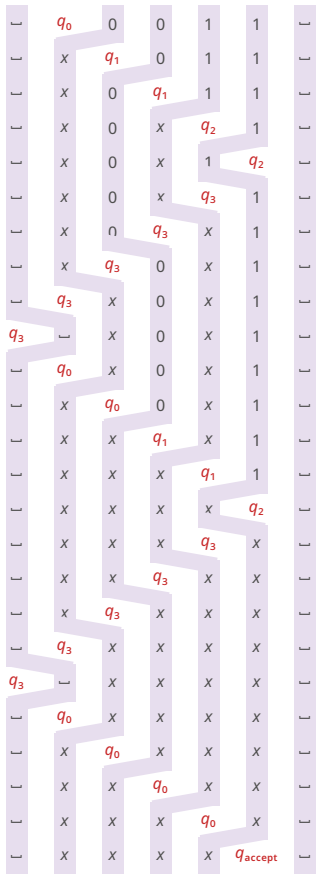
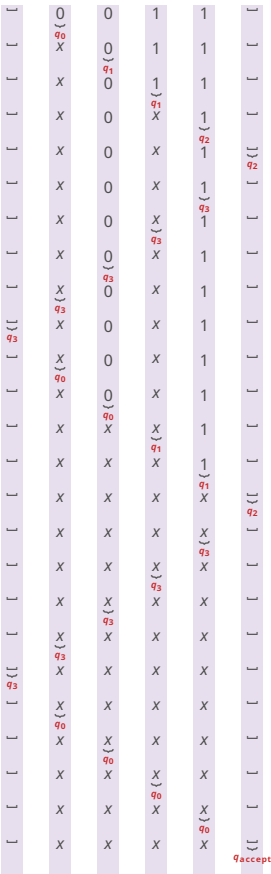
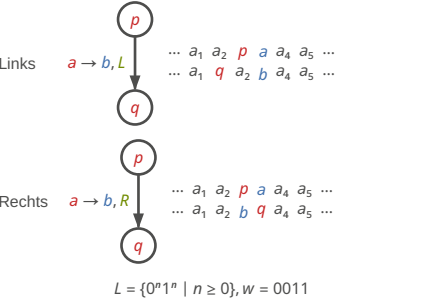
$\delta(p, a) = (q, b, L)$:

- Übergang möglich, wenn a unter dem Schreib- / Lese-Kopf
- Aktuelles Feld auf dem Band wird mit b überschrieben
- Kopfbewegung: L links, R rechts

PRÜFUNG

Bei der Letzten Aufgabe (Welche Sprache wird von M akzeptiert?) muss eine ausformulierte Begründung stehen.

BERECHNUNGSGESCHICHTE



VARIANTEN

NICHTDETERMINISTISCHE TM

Übergangsfunktion

Bei jedem Übergang maximal N verschiedene Möglichkeiten:
 $\delta : Q \times \Gamma \rightarrow P(Q \times \Gamma \times \{L, R\})$
 $\Rightarrow$ Mehrere mögliche Berechnungswege

Wort akzeptieren

$w \in L(M)$ wenn es einen Berechnungsweg gibt, der zu q_{accept} führt. (auch wenn es Wege gibt, die auf q_{reject} führen!)

Simulation auf Standard-TM

Verwende 3 Bänder:

- Arbeitsband
- Kopie von w
- Liste aller Folgen von Wahlmöglichkeiten

Simulation

- Kopiere w von Band 2 auf Band 1
- Führe TM aus auf Band 1 mit Wahlmöglichkeiten von B. 3
- Inkrementiere zur nächsten Folge auf Band 3

Simulierbar in $O(n^{\epsilon(n)}) = 2^{O(\epsilon(n))}$

VERSCHIEDENE BANDALPHABETE

Jedes andere Alphabet kann binär codiert werden.

Simulierbar in Zeit $O(t(n))$

MEHRSPURMASCHINE

Simulierbar in Zeit $O(t(n))$

MEHRBANDMASCHINE

Die unabhängige Bewegung der Schreib-/Leseköpfe auf einer n -Band Maschine kann mithilfe eines $2n$ -spurigen Bandes (Daten + Zeiger für Schreib-/Lesekopf) nachgebildet werden.

Simulierbar in Zeit $O(t(n)^2)$

EINSEITIG UNENDLICHES BAND

Beidseitig unendliches Band kann durch ein Alphabet doppelter Wortbreite realisiert werden, somit äquivalent.

TURING-ERKENNBARE SPRACHEN

Eine TM erkennt das Wort $w \in \Sigma^*$, wenn die Maschine auf dem Input w im Zustand q_{accept} anhält.

Sind L_1 und L_2 Turing-erkennbare Sprachen, dann sind auch der Durchschnitt $L_1 \cap L_2$ und die Vereinigungsmenge $L_1 \cup L_2$ Turing-erkennbar.

AUFZÄHLER

Aufzähler: TM mit einem Drucker, mit dem Wörter ausgedruckt werden können.

L ist **rekursiv aufzählbar** wenn es einen Aufzähler gibt, der alle Wörter aufzählen kann. L ist dann Turing-erkennbar.

ENTSCHEIDBARKEIT

Entscheider: eine Turing-Maschine, die auf jedem beliebigen Input anhält.
Turing-entscheidbare Sprache: L heisst **Turing-entscheidbar**, wenn es einen Entscheider M gibt mit $L = L(M)$.

ENTSCHEIDBARE PROBLEME

LEERHEITSPROBLEME

✓ Entscheidbar $E_{\text{DEA}} = \{\langle A \rangle \mid L(A) = \emptyset\}$

✓ Entscheidbar $E_{\text{CFG}} = \{\langle G \rangle \mid L(G) = \emptyset\}$

Wie: Chomsky-Normalform

✗ Entscheidbar $E_{\text{TM}} = \{\langle M \rangle \mid L(M) = \emptyset\}$

Denn Reduktion möglich: $A_{\text{TM}} \leq E_{\text{TM}}$

GLEICHHEITSPROBLEME

✓ Entscheidbar $\text{EQ}_{\text{DEA}} = \{\langle A_1, A_2 \rangle \mid L(A_1) = L(A_2)\}$

Wie: Vergleich der minimalen Automaten, oder Leerheitsproblem bei: $L(A_1) \triangle L(A_2)$

✗ Entscheidbar $\text{EQ}_{\text{CFG}} = \{\langle G_1, G_2 \rangle \mid L(G_1) = L(G_2)\}$

Denn Es gibt eine Reduktion $\text{ALL}_{\text{CFG}} \leq \text{EQ}_{\text{CFG}}$. Da ALL_{CFG} nicht entscheidbar, auch EQ_{CFG} nicht entscheidbar.

✗ Entscheidbar $\text{EQ}_{\text{TM}} = \{\langle M_1, M_2 \rangle \mid L(M_1) = L(M_2)\}$

Denn EQ_{PDA} nicht entscheidbar

AKZEPTANZPROBLEME

✓ Entscheidbar $A_{\text{DEA}} = \{\langle A, w \rangle \mid w \in L(A)\}$

Wie: Regex-Engines simulieren beliebige DEAs auf beliebigen Input-Wörtern w

✓ Entscheidbar $A_{\text{CFG}} = \{\langle G, w \rangle \mid w \in L(G)\}$

Wie: CYK, deterministischer Parse-Algorithmus

✓ Entscheidbar $A_{\text{CFG}} = \{\langle G, w \rangle \mid \varepsilon \in L(G)\}$

Wie: wenn CNF Regel $S \rightarrow \varepsilon$ enthält

✗ Entscheidbar $A_{\text{TM}} = \{\langle M, w \rangle \mid w \in L(M)\}$

Denn Halteproblem

ALLE WÖRTER PRODUZIEREN

✗ Entscheidbar $\text{ALL}_{\text{CFG}} = \{\langle G \rangle \mid G \text{ produziert } \Sigma^*\}$

Denn (Gegenteil von E Problem mit TM) $A_{\text{TM}} \leq \text{ALL}_{\text{PDA}}$

✗ Entscheidbar $\text{ALL}_{\text{PDA}} = \{\langle P \rangle \mid P \text{ akzeptiert } \Sigma^*\}$

Denn Reduktionsabbildung der Gegenteil akzeptiert

WEITERE

✗ Entscheidbar $\text{HALT}_{\text{TM}} = \{\langle M, w \rangle \mid M \text{ hält auf Input } w\}$

Denn Auf A_{TM} reduziert $\rightarrow$ nicht entscheidbar

SATZ VON RICE

Ist P eine nichttriviale Eigenschaft Turing-erkennbarer Sprachen, dann ist P_{TM} nicht entscheidbar.

Eine Eigenschaft P Turing-erkennbarer Sprachen heisst **nichttrivial**, wenn es zwei Turing-Maschinen M_1 und M_2 gibt, wobei M_1 die Eigenschaft hat und M_2 nicht.

Beispiel 13: Links-kürzbar

Man sagt, die Sprache sei links-kürzbar, wenn man von einem Wort ein beliebiges Anfangsstück entfernen kann und das verkürzte Wort immer noch ein Wort der Sprache ist. Also zum Beispiel $ASDF \in L \Rightarrow SDF, DF, F, \varepsilon \in L$. Wie könnte man ein Programm aufbauen, welches immer anhält und mit welchem man andere Programme analysieren kann, ob deren akzeptierte Sprache links-kürzbar ist?

Die Eigenschaft einer Sprache, links-kürzbar zu sein, ist eine nichttriviale Eigenschaft. Die Liste der Wörter in der Aufgabenstellung bildet eine links-kürzbare Sprache, die Sprache bestehend nur aus dem Wort BIBER hat diese Eigenschaft nicht. Der Satz von Rice besagt jetzt, dass man kein Programm schreiben kann, welches entscheiden könnte, ob die akzeptierte Sprache die Eigenschaft hat, links-kürzbar zu sein.

Satz von Rice (R) 1 Punkt, Eigenschaft (E) 1 Punkt, zwei Sprachen (Z) 1 Punkt, Eigenschaft ist nicht trivial (T) 1 Punkt, Schlussfolgerung nicht entscheidbar (N) 2 Punkte.

LÖSUNGSANLEITUNG EINER PRÜFUNGSFRAGE:

- Nichttriviale Eigenschaft P aufschreiben
- Die beiden Sprachen $L(M_1)$ und $L(M_2)$ bilden (meistens kann man die leere Menge oder Σ^* als eine dieser Sprachen verwenden)
- Gibt es ein Programm, welches beide Sprachen erkennen kann? Sind beide Sprachen Turing erkennbar?
- Dann besagt der Satz von Rice, dass die Sprache nicht entscheidbar ist

Beispiel 14: Datum

Ein häufig wiederkehrende Aufgabe in der Softwareentwicklung ist die Beurteilung, ob eine Zeichenkette ein gültiges Datum ist. Daher könnte es nützlich sein, ein Tool zur Verfügung zu haben, welches ein Programm analysieren kann und eine Aussage liefern kann, ob das Programm nur korrekte Daten akzeptiert. Gibt es ein solches Tool?

Nein. Dies ist das Entscheidungsproblem für die Eigenschaft DATES_{TM} , wobei die Eigenschaft DATES besagt, dass die Sprache nur aus korrekten Daten besteht. Diese Eigenschaft ist nicht trivial, denn

$L_1 = \{2024 - 07 - 19\}$ hat die Eigenschaft DATES ,
 $L_2 = \{blubb\}$ hat die Eigenschaft nicht.

Beide Sprachen können von einer Turing-Maschine erkannt werden. Nach dem Satz von Rice ist DATES_{TM} nicht entscheidbar.

Eigenschaft DATES (P) 1 Punkt, zwei Sprachen (L) 1 Punkt, Eigenschaft ist nicht trivial (T) 1 Punkt, Anwendung des Satzes von Rice (R) 2 Punkte.

NP

Eine Sprache L gehört zur Klasse NP , wenn L von einer nichtdeterministischen TM in **polynomieller Zeit** entschieden werden kann

POLYNOMIELLE VERIFIZIERER

Ein **polynomieller Verifizierer** für die Sprache L ist eine TM V , so dass es für jedes $w \in \Sigma^*$ ein Wort c (das Lösungszertifikat) gibt, für das gilt

$w \in L \Leftrightarrow V$ akzeptiert $\langle w, c \rangle$

Die Laufzeit von V ist polynomiell in $|w|$.

Eine Sprache ist genau dann in **NP**, wenn sie in polynomieller Zeit **verifiziert** werden kann.

Bei der Prüfung Entscheidbarkeit und Zertifikat erwähnen!

Beispiel 15: Square killer

Square Killer ist eine Variante von Sudoku, die auf einem $n^2 \times n^2$ Spielfeld gespielt wird. Die normalen Sudoku-Regeln gelten, zusätzlich muss aber die Zahl der Ziffern in einem zusammenhängenden grauen Gebiet («Käfig») eine Quadratzahl sein.

Das Problem ist natürlich entscheidbar, man kann die endlich vielen möglichen Zahlenverteilungen durchprobieren und die Regeln überprüfen.

Die Fragestellung ist gleichbedeutend mit der Frage, ob es einen polynomiellen Verifizierer gibt.

Für einen polynomiellen Verifizierer brauchen wir ein Lösungszertifikat, wir nehmen dafür die Zahlen, die in den Feldern stehen. Folgende Verifikationsschritte sind noch nötig:

Schritt	Aufwand
1 Sudoku-Verifizierer	polynomiell
2 Für jedes Feld überprüfen, ob die Summe der Zahlen im gleichen Käfig eine Quadratzahl ist.	$O(n^4 \cdot n^4)$
Total	polynomiell

Entscheidbarkeit (E) 1 Punkt, Verifizierer (V) 1 Punkt, Zertifikat (Z) 1 Punkt, bestehende Sudoku Regeln sind in polynomieller Zeit verifizierbar (S) 1 Punkt, Verifikation von zwei zusätzlichen Regel (R) 1 Punkt, Laufzeit polynomiell (L) 1 Punkt.

m-SUDOKU

Entscheidbar? Ja: $n = m^4$ = Anzahl Felder
Zertifikat: Vollständig ausgefülltes Sudoku
Vorgehen:

Schritt	Aufwand
1 Jedes Feld: Zeichen kommt auf der Zeile nicht mehr vor	$O(n^4 \cdot n^2)$
2 Jedes Feld: Zeichen kommt in der Spalte nicht mehr vor	$O(n^4 \cdot n^2)$
3 Jedes Feld: Zeichen kommt im Unterquadrat nicht vor	$O(n^4 \cdot n^2)$
Total	$O(n^6)$

NURKABE

Entscheidbar? Ja: $n = uv$ = Anzahl Felder. Alle Belegungen des Spielfeldes mit schwarzen Feldern können überprüft werden, ob sie die Regeln einhalten.
Zertifikat: Liste der schwarzen Felder
Vorgehen:

Schritt	Aufwand
1 Für jedes Zahlfeld ($O(n)$) verwendet man einen Markieralgorithmus, der alle zum Gebiet dieses Zahlfeldes gehörenden weissen Felder bestimmt ($O(n)$ Durchgänge mit Aufwand $O(n)$).	$O(n^3)$
2 Trifft dieser Algorithmus auf ein weiteres Zahlfeld, ist der erste Teil von Regel 1 verletzt ($\rightarrow q_{\text{reject}}$).	$O(n)$
3 Weicht die Zahl der gefundenen weissen Felder vom Inhalt des Zahlfeldes ab, ist der zweite Teil von Regel 1 verletzt ($\rightarrow q_{\text{reject}}$).	$n \cdot O(n)$
4 Ebenfalls mit einem Markieralgorithmus wird überprüft, ob das schwarze Gebiet zusammenhängend ist.	$O(n^2)$
5 Für jedes schwarze Feld wird überprüft, ob es Teil eines 2×2 -Tümpels ist.	$O(n)$
Total	$O(n^3)$

DAMEN

TODO:

REDUKTION

Polynomielle Reduktion: $A \leq_p B$. Lies: A ist polynomiell leichter entscheidbar als B .

NP-VOLLSTÄNDIG

Reduktion auf NP-Vollständiges Problem

TURING-VOLLSTÄNDIG

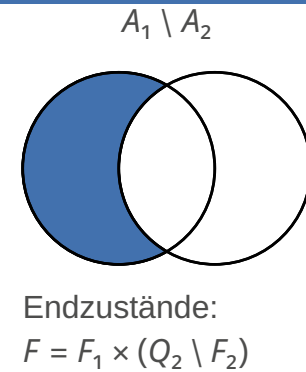
Eine Sprache A heisst eine **Programmiersprache**, wenn es eine Abbildung $c : A \mapsto \Sigma^*$ gibt, wobei $c(w)$ ein Programm für eine universelle Turing-Maschine ist.

Eine Programmiersprache heisst **Turing-vollständig**, wenn in ihr jede beliebige Turing-Maschine simuliert werden kann.

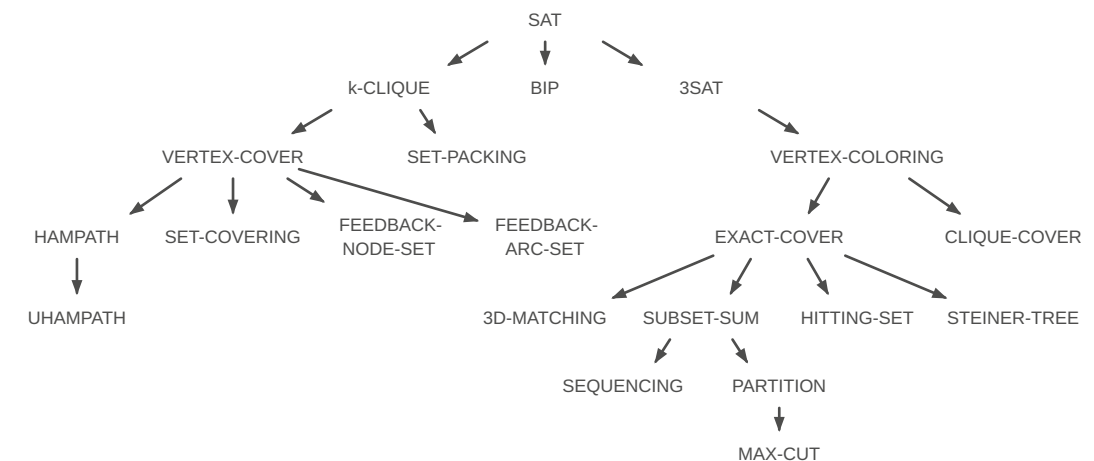
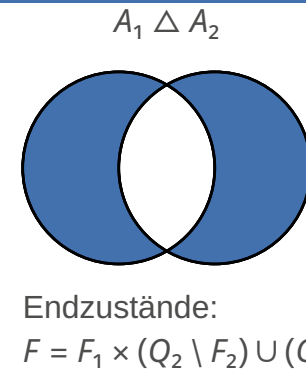
LOGIK

Zu einer ganzzahligen Matrix C und einem ganzzahligen Vektor d , ist ein binärer Vektor x zu finden mit $C \cdot x = d$

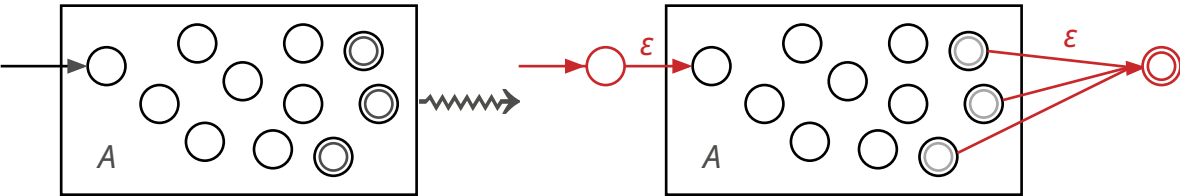
DIFFERENZMENGE $L(A_1) \setminus L(A_2)$



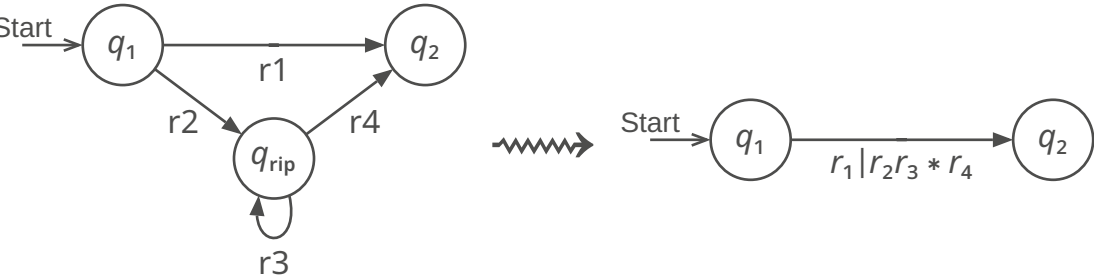
SYMMETRISCHE DIFFERENZ $L(A_1) \triangle L(A_2)$



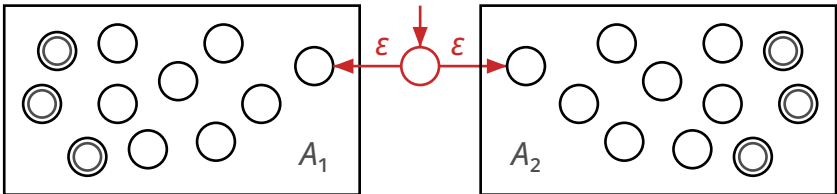
Keine Übergänge nach q_0 und nur ein Akzeptierzustand



Nach Entfernen aller Zwischenzustände $q_{rip} \in Q$ bleibt ein regulärer Ausdruck r von A

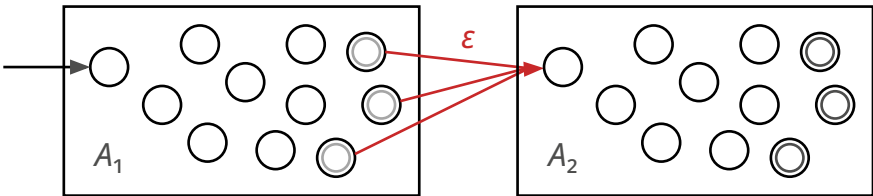


$$L = L_1 \cup L_2$$
$$= L(A_1) \cup L(A_2)$$
$$= L(A_1) \mid L(A_2)$$



VERKETTUNG

$$L = L_1 L_2$$
$$= L(A_1) L(A_2)$$
$$= \{w_1 w_2 \mid w_i \in L_i\}$$



*-OPERATION

$$L^* = \{\epsilon\} \cup L \cup L^2 \cup \dots$$
$$= \bigcup_{k=0}^{\infty} L^k$$

